

Basic Python Course — Curriculum, Schedule and Budget

A complete beginner's programming course for **Class 12 Science students**, taking them from never having written a line of code to building a working Quiz Game of their own.

Format: 15 classes of 45 minutes each, delivered across a **maximum of 10 days**. The college can run one or two classes on any given day — the material is sized so that each lesson file is exactly one class, so the timetable can flex without changing the content.

The project: Every concept is taught with an eye on the final build. In the last two classes the whole group constructs a terminal **Quiz Game** — multiple-choice questions across Physics, Chemistry, Biology, Maths and Python, with shuffled rounds, category selection, scoring, grading and a high-score table that survives after the program closes.

Requirements: Python 3.12 or newer and Visual Studio Code. Nothing else. No internet connection is needed during class, no accounts are created and no third-party packages are installed.

Instructor

Er. Prabin Karki — Senior Software Engineer
B.E. in Computer Science and Engineering · Lalitpur, Nepal
prabin-karki.com.np · prabinkarki643@gmail.com

A software engineer with **over 8 years of professional experience**, currently Senior Software Engineer at ArBA Development Studio, where he has worked since 2018. He has delivered more than 15 production systems — from startup products to national-scale government platforms, including a Border and Immigration Portal and Hamro Blood Bank.

He works across Python, TypeScript and JavaScript, with a stack spanning React and React Native, Node.js and NestJS, PostgreSQL and MongoDB, and AWS deployment. Alongside building, he leads cross-functional teams, runs code reviews and mentors junior developers — the habit of explaining *why* code works, not just fixing it, is what this course is built on.

Experience	8+ years, professional software development
Current role	Senior Software Engineer, ArBA Development Studio (2018–present)
Languages	Python, TypeScript, JavaScript
Specialisms	Web and mobile applications, cloud infrastructure, technical mentoring
Portfolio	prabin-karki.com.np
Email	prabinkarki643@gmail.com

Course At A Glance

Total classes	15
Class length	45 minutes (≈ 32 min teaching + 13 min supervised practice)
Total teaching time	11 hours 15 minutes
Delivery window	10 days maximum, 1–2 classes per day
Level	Absolute beginner — no prior coding experience assumed
Language	Python 3.12+, standard library only
Final project	Terminal Quiz Game
Instructor	Er. Prabin Karki

Suggested 10-Day Schedule

The split below totals 15 classes across 10 days. It is a suggestion — any arrangement of one or two classes per day works, provided the classes run in order.

Day	Classes	Focus
1	1 + 2	Setup, first program, variables and data types
2	3 + 4	User input, type conversion, all the operators
3	5	Decisions — <code>if</code> , <code>elif</code> , <code>else</code>
4	6 + 7	Loops — <code>for</code> , <code>range()</code> , <code>while</code> , <code>break</code> , <code>continue</code>
5	8	Strings in depth
6	9 + 10	Lists, tuples and sets
7	11	Dictionaries
8	12 + 13	Functions, modules, error handling, files
9	14	Mini project — building the Quiz Game
10	15	Mini project — finishing the game, and what next

Phase 1 — Getting Started (Classes 1–3)

Class 1 · Introduction to Python and Setup → 01-introduction-and-setup.md

- ☐ What a program is; why Python; installing Python 3 and VS Code
- ☐ Running a program from the editor and from the terminal
- ☐ `print()`, quotes, printing several items, comments with `#`
- ☐ Reading a first error message — `SyntaxError` and `NameError`

Exercise: Write a personal profile program and deliberately break it three ways

Class 2 · Variables and Data Types → 02-variables-and-data-types.md

- ☐ Variables as labelled boxes; why `=` means "put into", not "equals"
- ☐ `int`, `float`, `str`, `bool` and checking with `type()`
- ☐ Naming rules and snake_case conventions; reserved words
- ☐ Reassignment, multiple assignment and the one-line swap

Exercise: Build a student record with marks, total and average

Class 3 · Input, Output and Type Conversion → 03-input-output-and-type-conversion.md

- ☐ `input()` and why it **always** returns a string
- ☐ Converting with `int()`, `float()` and `str()`; the `ValueError`
- ☐ f-strings, decimal places with `:.2f`, and `round()`

Exercise: Build a Celsius / Fahrenheit / Kelvin temperature converter

Phase 2 — Logic and Control Flow (Classes 4–7)

Class 4 · Operators and Expressions → 04-operators-and-expressions.md

- ☐ All seven arithmetic operators, including `//`, `%` and `**`
- ☐ Shortcut assignment with `+=`; comparison operators and `bool` results
- ☐ `and`, `or`, `not`, truth tables and chained comparisons
- ☐ Operator precedence, and why `^` is not a power operator

Exercise: Split a three-digit number into its digits using only `//` and `%`

Class 5 · Making Decisions → 05-decisions-if-elif-else.md

- ☐ `if`, `else`, `elif`; the colon and 4-space indentation as syntax
- ☐ Why the **order** of an `elif` chain decides whether it is correct
- ☐ Nested conditions, and when `and` reads better
- ☐ Truthiness — the values Python counts as false

Exercise: Write a leap-year checker and a quadratic-discriminant classifier

Class 6 · Loops Part 1 — `for` and `range()` → `06-for-loops-and-range.md`

- ☐ `for` loops over ranges and strings
- ☐ `range()` in all three forms; why the stop value is excluded
- ☐ The accumulator pattern — totals, counts, products, maximums
- ☐ Combining a loop with an `if` to count and filter

Exercise: Multiplication tables and a ten-year compound interest table

Class 7 · Loops Part 2 — `while`, `break`, `continue` → `07-while-loops-break-continue.md`

- ☐ `while` loops and choosing between `for` and `while`
- ☐ Infinite loops, the three things every `while` needs, and `Ctrl + C`
- ☐ `break` to leave early; `continue` to skip a turn
- ☐ The `while True` input-validation pattern; nested loops and patterns

Exercise: Build a five-guess number guessing game with hints

Phase 3 — Working With Data (Classes 8–11)

Class 8 · Strings in Depth → `08-strings.md`

- ☐ Indexing forwards and backwards; slicing; `[::-1]` to reverse
- ☐ `.upper()`, `.lower()`, `.strip()`, `.find()`, `.replace()`, `.split()`, `.join()`
- ☐ Why strings are **immutable** and methods return a new value
- ☐ f-string alignment (`:<15`, `:>5`, `:^20`) for tidy columns

Exercise: Build a text analyser — word count, vowels, longest word, palindrome test

Class 9 · Lists → `09-lists.md`

- ☐ Creating, indexing and slicing lists; lists are **mutable**
- ☐ `.append()`, `.insert()`, `.remove()`, `.pop()`, `.clear()`
- ☐ `len()`, `sum()`, `max()`, `min()` replacing whole loops
- ☐ `.sort()` versus `sorted()`; looping with and without the index

Exercise: Rebuild the Class 6 marks analyser with a working bar chart and ranking

Class 10 · Tuples and Sets → `10-tuples-and-sets.md`

- ☐ Tuples and immutability; the trailing comma; when to choose one
- ☐ Unpacking — the one-line swap, multiple return values, `for name, marks in ...`
- ☐ Sets: no duplicates, no order; removing duplicates from a list
- ☐ Union, intersection and difference — Venn diagrams as code

Exercise: Compare two groups' subject choices using set operations

Class 11 · Dictionaries → 11-dictionaries.md

- ☐ Key-value pairs; `d[key]` versus the safer `d.get(key, default)`
- ☐ Adding, updating, deleting; `del` and `.pop()`
- ☐ Looping with `.keys()`, `.values()` and `.items()`
- ☐ The counting pattern `counts[x] = counts.get(x, 0) + 1`
- ☐ Nested dictionaries and a **list of dictionaries** — the Quiz Game's shape

Exercise: Build a student database with per-subject and per-student statistics

Phase 4 — Functions, Files and Safety (Classes 12–13)

Class 12 · Functions and Modules → 12-functions-and-modules.md

- ☐ `def`, calling, parameters versus arguments, keyword and default arguments
- ☐ `return` — and the crucial difference between returning and printing
- ☐ Returning several values as a tuple; writing docstrings
- ☐ Local versus global scope, and why passing values in is better
- ☐ `import math` and `import random`; `randint`, `choice`, `shuffle`

Exercise: Refactor the Class 5 grade calculator into four clean functions

Class 13 · Errors, Exceptions and Files → 13-errors-files-and-exceptions.md

- ☐ Syntax errors versus exceptions; reading a traceback bottom-up
- ☐ `try` / `except` / `else` / `finally`; catching the right exception
- ☐ The unbreakable input pattern: `while True` + `try` / `except` + `return`
- ☐ `with open(...)`, modes `"r"`, `"w"`, `"a"`, and why `"w"` is dangerous
- ☐ Reading data back and guarding every assumption

Exercise: Build a marks store that saves to disk and survives a corrupted file

Phase 5 — Mini Project: the Quiz Game (Classes 14–15)

From here the whole class builds **one program** — a Quiz Game they can play and share. Every class from 1 to 13 shows up in it.

Class 14 · Building the Quiz Game → 14-project-quiz-game-part-1.md

- ☐ Planning a program on paper before typing; splitting work into functions
- ☐ The question bank as a list of dictionaries; storing the answer as an index
- ☐ Splitting a program across two files with `from questions import QUESTIONS`

- ☐ Validating the player's answer so nothing can crash the game
- ☐ The scoring loop, percentage and grade

Exercise: Grow the question bank and add a two-option true/false question

Class 15 · Finishing the Game and What Next → 15-project-quiz-game-part-2.md

- ☐ Shuffling with `random.shuffle()` and why the list must be copied first
- ☐ Building the category menu **from the data**, so new categories appear free
- ☐ Saving high scores with mode `"a"` ; reading them back defensively
- ☐ Sorting a list of tuples; the replay loop
- ☐ An honest map of what was not covered, and where Python goes next

Exercise: Add negative marking, personal bests, difficulty levels and a timer

What Students Will Be Able To Do

By the end of the course, every student can:

- Install Python, write a program in VS Code and run it from the terminal
- Store data in variables and choose the right type for the job
- Take input from a user, convert it safely and display formatted results
- Write conditions and loops to control what a program does and how often
- Use all four core collections — lists, tuples, sets and dictionaries — and explain when each is appropriate
- Break a problem into functions that each do one job well
- Read a traceback, handle errors gracefully and save data to a file
- Build, debug and extend a complete program of several hundred lines

Just as importantly, they will have read enough error messages to stop being frightened by them.

What Is Not Covered

Stated plainly so that expectations are accurate. This is a fundamentals course, and 15 classes is 15 classes.

- Object-oriented programming — classes, objects, inheritance
- Third-party libraries — NumPy, pandas, Matplotlib, Requests
- Web frameworks (Django, Flask), databases and APIs
- GUI applications (Tkinter)
- Virtual environments, `pip`, packaging and deployment
- Advanced language features — comprehensions, decorators, generators, recursion

Class 15 closes with a clear map of these topics and recommended next steps, so students leave knowing exactly what they have learned and what comes after.

Requirements

From the college:

- A computer lab with one PC per student, or students bringing their own laptops
- **Python 3.12 or newer** installed on every machine before Day 1
- **Visual Studio Code** with the official Python extension installed
- A projector or large display for live coding
- A whiteboard

Not required: internet access during class, student email accounts, software licences, or any paid tools. Everything taught runs on a plain Python installation.

Please note: The single most common cause of a lost first class is Python being installed without **"Add python.exe to PATH"** ticked on Windows. Class 1 covers the fix, but installing correctly beforehand saves everyone twenty minutes.

Budget

Fee Structure

Item	Rate	Classes	Amount
Instructor fee	Rs. 1,500 / class	15	Rs. 22,500
Total course cost		15	Rs. 22,500

The fee is charged **per class delivered**, not per student. Batch size does not affect the cost, and the total is unchanged whether the college runs one or two classes on any given day.

What Is Included

- All 15 classes taught in person
- Complete written lesson notes for every class, provided to the college
- Every code example tested and runnable, with expected output shown
- 45+ practice exercises with the material needed to solve them
- The full reference project source code
- Guidance for students on how to continue after the course

What Is Not Included

- Lab hardware, electricity and internet connectivity
- Printing or photocopying of notes
- Student certificates, if the college wishes to issue them
- Travel outside the Kathmandu valley
- Any assessment or examination beyond the in-class exercises

Contact

To discuss scheduling, batch size or any adjustment to the syllabus, please get in touch.

Er. Prabin Karki

Email: prabinkarki643@gmail.com

Portfolio: prabin-karki.com.np

Course content © Er. Prabin Karki. Free to use for educational purposes with attribution.